

数据存储与管理

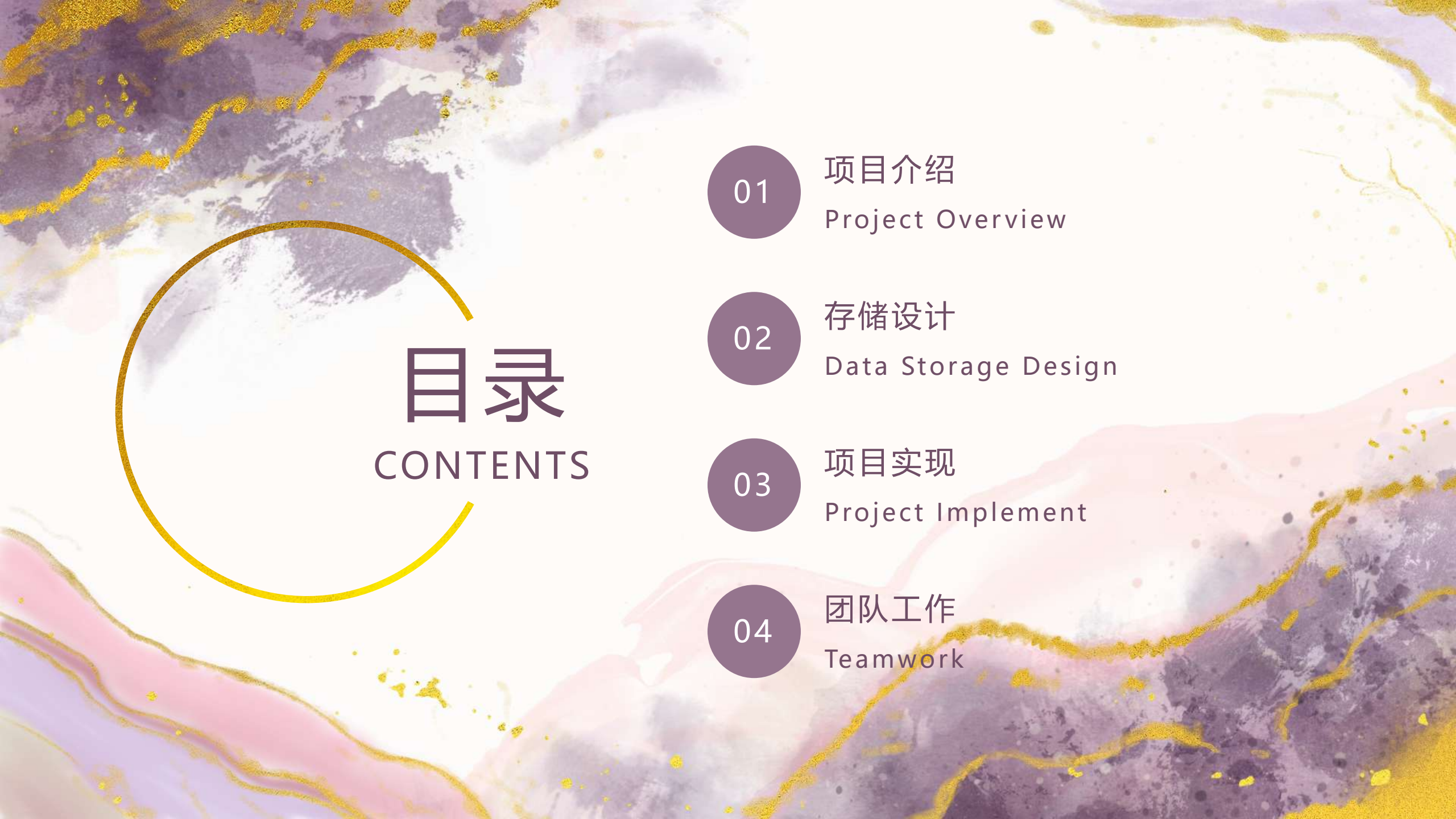
Data Storage & Management

Team:

2351299 王雷

2354185 周达

2351129 黄景胤



目录

CONTENTS

01

项目介绍

Project Overview

02

存储设计

Data Storage Design

03

项目实施

Project Implement

04

团队工作

Teamwork



01

项目介绍

Project Overview

数据库类型

openGauss (关系型)

- **核心定位：**交互式查询
- 📦 **数据模型：**结构化星型模型
- 🔍 **典型查询：**精确筛选、排序、聚合
- ⚙️ **优化关键：**索引、冗余、非规范化

Hive (分布式)

- **核心定位：**离线批量分析
- 📦 **数据模型：**半结构化文件
- 🔍 **典型查询：**全表扫描、趋势统计
- ⚙️ **优化关键：**分区、列式存储

Neo4j (图数据库)

- **核心定位：**复杂关系挖掘
- 📦 **数据模型：**节点与关系图
- 🔍 **典型查询：**多跳路径、社群发现
- ⚙️ **优化关键：**关系建模

项目构建

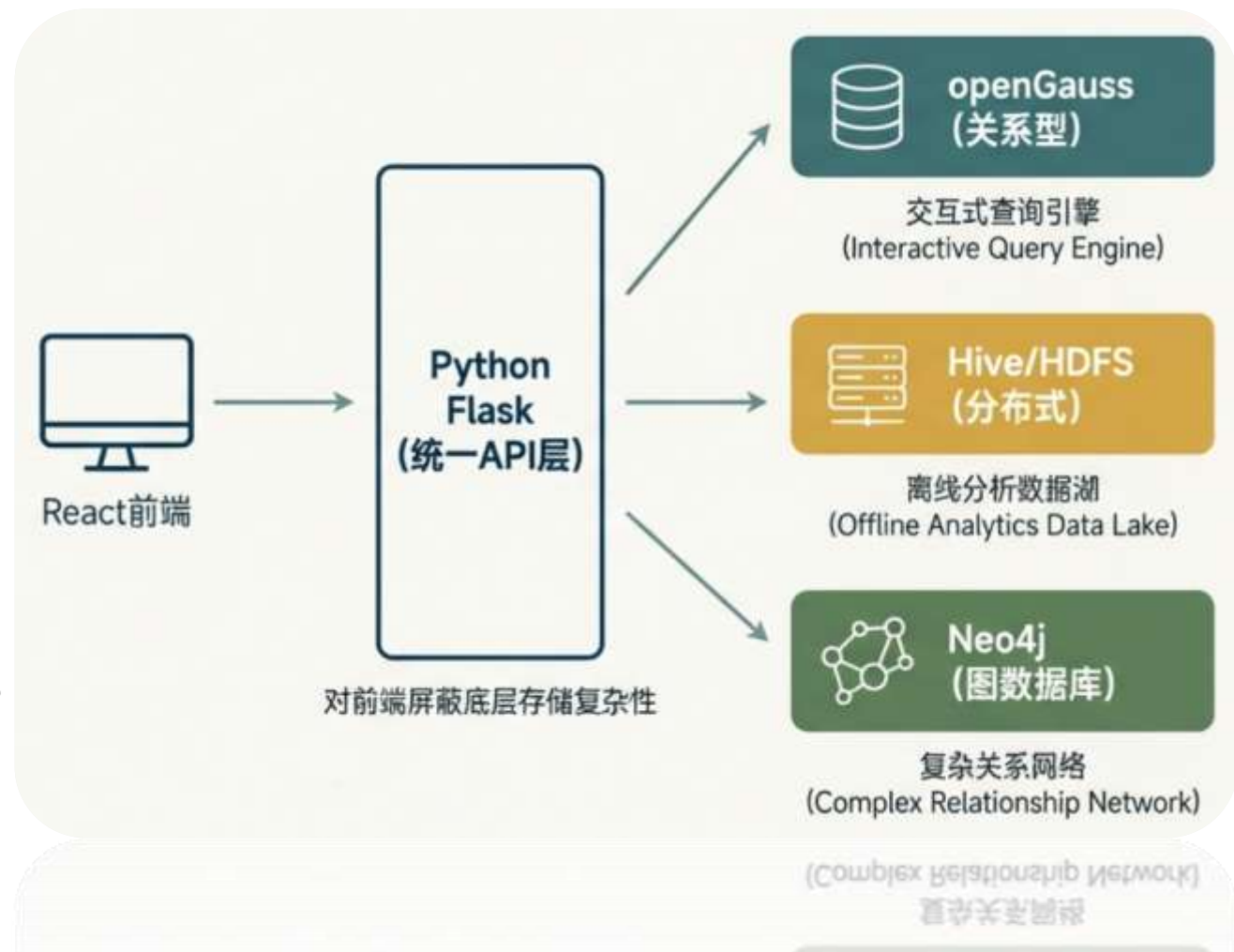
使用MVC三层架构

将视图层、业务逻辑层和控制层分开

前端使用React进行用户页面显示



后端使用Flask框架对用户的请求进行处理，并与数据库进行交互



项目部署

数据库:

部署在远程云服务器，通过公网IP进行访问

✔ 允许	1	自定义 TCP	IPv4	⚠ 任何位置 (0.0.0.0/0)	端口	7687/7687
✔ 允许	1	自定义 TCP	IPv4	⚠ 任何位置 (0.0.0.0/0)	端口	7077/7077
✔ 允许	1	自定义 TCP	IPv4	⚠ 任何位置 (0.0.0.0/0)	端口	7474/7474
✔ 允许	1	自定义 TCP	IPv4	⚠ 任何位置 (0.0.0.0/0)	端口	10000/10000
✔ 允许	1	PostgreSQL 访问 TCP:5432/5432	IPv4	❗ 任何位置 (0.0.0.0/0)	端口	PostgreSQL(5432)

前端页面与后端服务:

在本地进行部署，访问远程的数据库连接

```
> data-warehouse@0.0.0 dev
> vite

ROLLDOWN-VITE v7.2.5 ready in 262 ms

→ Local: http://localhost:3000/
→ Network: use --host to expose
→ press h + enter to show help
```

```
Starting Movie Data Warehouse Backend...
Host: 0.0.0.0
Port: 5000
Debug: True
```




02

存储设计

Data Storage Design

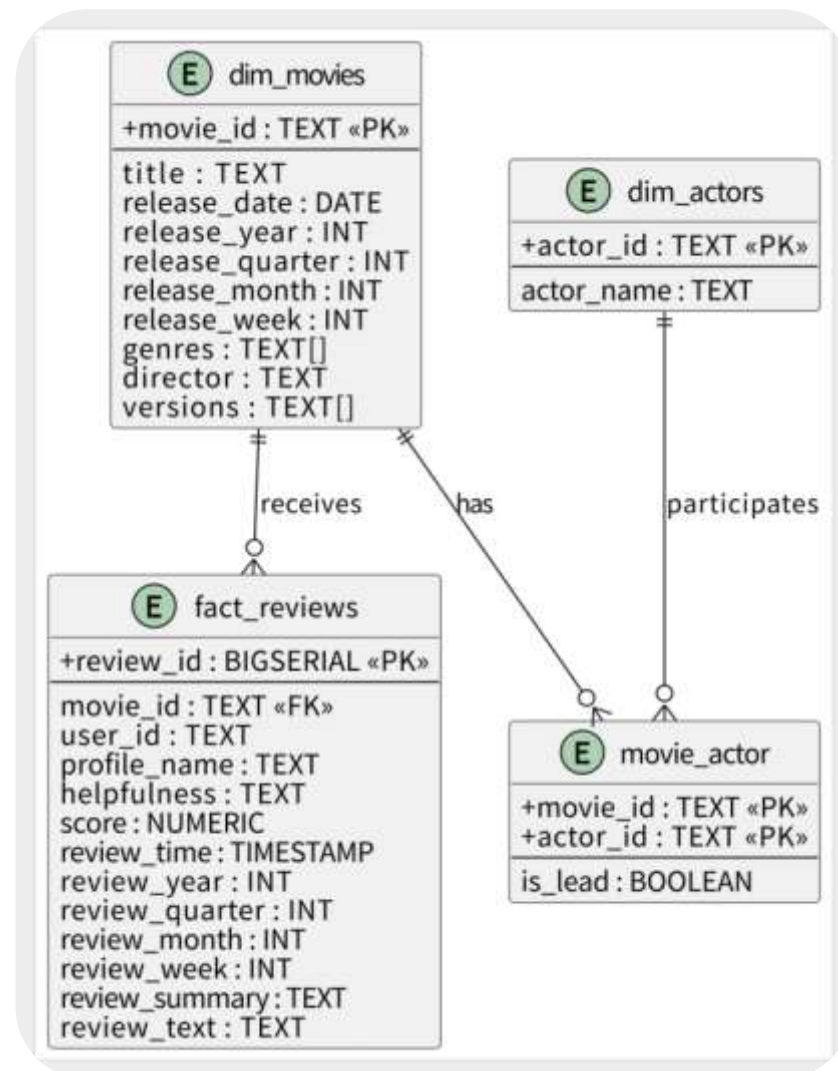
关系型数据库

逻辑设计:

用四张表对数据进行联系和存储

性能优化:

- 时间维度优化
- 索引优化
- 冗余字段设计
- 事实表与维度表分离



关系型数据库

时间维度优化:

- 对原始日期字段进行冗余设计, 将电影上映时间和评论时间拆分为年、季度、月和周等多个字段。
- 使在执行按时间统计数量时, 数据库无需在查询中使用日期函数进行计算, 减少开销

索引优化:

- 为电影维度表中的时间字段建立了单列和组合索引, 同时在评论表中对movie_id 和 score 字段建立索引。
- 当按照电影、评分或时间维度进行统计时, 数据库能够快速定位对应数据行, 无需扫描全表, 从而显著减少查询延迟

-- 索引及时间维度索引

```
CREATE INDEX idx_fact_movie ON fact_reviews(movie_id);
CREATE INDEX idx_fact_score ON fact_reviews(score);
CREATE INDEX idx_movie_release_year ON dim_movies(release_year);
CREATE INDEX idx_movie_release_quarter ON dim_movies(release_year, release_quarter);
CREATE INDEX idx_movie_release_month ON dim_movies(release_year, release_month);
CREATE INDEX idx_movie_release_week ON dim_movies(release_year, release_week);
CREATE INDEX idx_review_year ON fact_reviews(review_year);
CREATE INDEX idx_review_quarter ON fact_reviews(review_year, review_quarter);
CREATE INDEX idx_review_month ON fact_reviews(review_year, review_month);
CREATE INDEX idx_review_week ON fact_reviews(review_year, review_week);
```

```
CREATE INDEX idx_fact_review_year ON fact_reviews(movie_id, review_year);
```

```
CREATE INDEX idx_fact_review_quarter ON fact_reviews(movie_id, review_year, review_quarter);
```

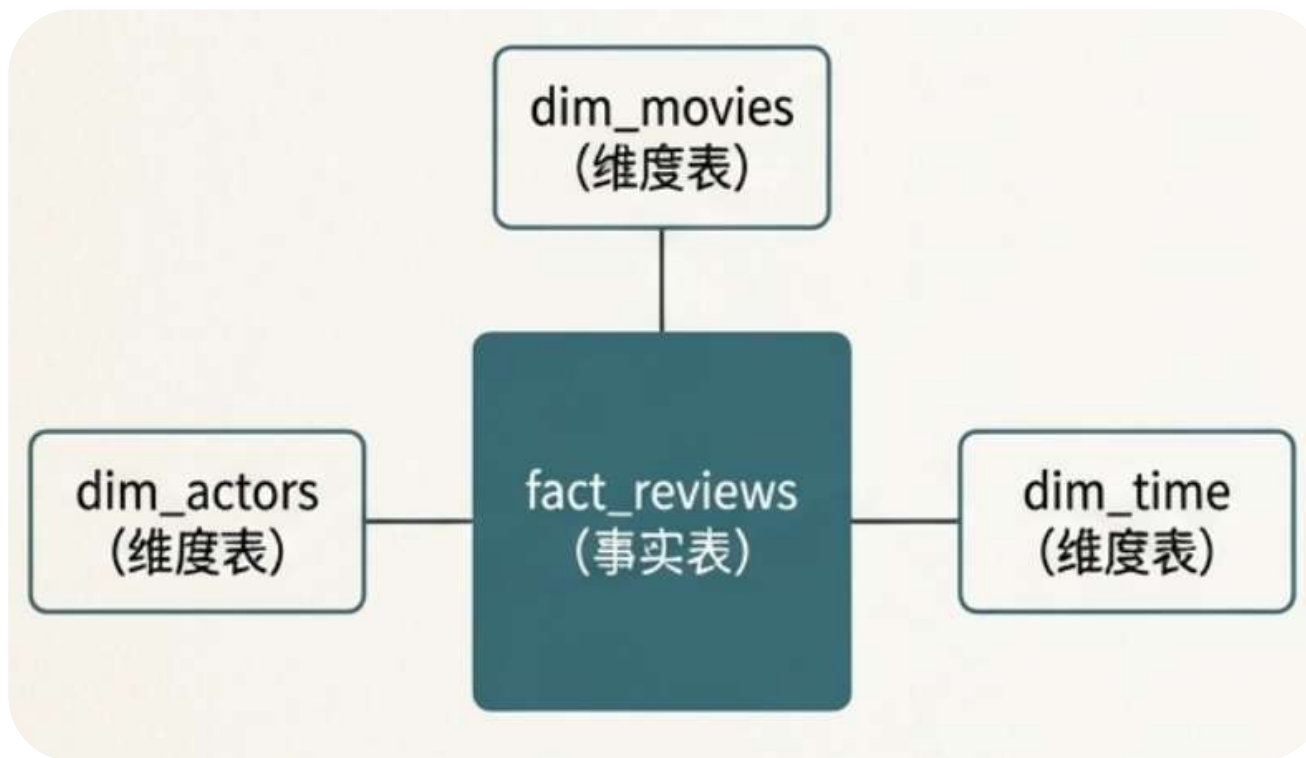
关系型数据库

冗余字段设计:

- 电影类别和版本信息被存储为数组类型, 这样无需额外建立关联表, 减少了多表 Join 带来的性能开销
- 电影-演员关系单独存储在关联表 movie_actor 中, 保持了数据的规范化, 避免冗余数据更新带来的复杂性

事实表和维度表分离:

- 事实表与维度表的分离设计, 使得历史评论数据不会因为维度信息的更新而受影响, 保证数据一致性



分布式存储

表设计:

用两张表分别存储电影和评论的信息

```
CREATE EXTERNAL TABLE IF NOT EXISTS reviews_clean_amazon (  
  asin STRING COMMENT 'product/productId',  
  user_id STRING COMMENT 'review/userId',  
  profile_name STRING COMMENT 'review/profileName',  
  helpfulness STRING COMMENT 'review/helpfulness, e.g. 2/3',  
  score DOUBLE COMMENT 'review/score',  
  review_unix BIGINT COMMENT 'review/time (unix)',  
  review_summary STRING COMMENT 'review/summary',  
  review_text STRING COMMENT 'review/text'  
)  
PARTITIONED BY (year INT, month INT)  
STORED AS PARQUET  
LOCATION '/warehouse/clean/reviews_amazon/';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS movies_meta_dw  
  asin STRING,  
  title STRING,  
  release_date DATE,  
  genres ARRAY<STRING>,  
  director ARRAY<STRING>,  
  actors ARRAY<STRING>,  
  starring ARRAY<STRING>,  
  versions ARRAY<STRING>,  
  source_files ARRAY<STRING>,  
  release_year INT, -- 时间维度字段 (用于分区)  
  release_quarter INT,  
  release_month INT,  
  release_week INT  
)  
PARTITIONED BY (  
  p_year INT,  
  p_month INT  
)  
STORED AS PARQUET  
LOCATION '/warehouse/clean/movies_meta_dw/';
```

Parquet 列式存储

行式存储

ID	User	Content	Rating	Date
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

列式存储

ID	User	Content	评分	Date
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

采用Parquet列式存储，分析查询（如统计所有评论的平均分）只需读取“评分”这一列，极大降低了I/O开销。

采用Parquet列式存储，分析查询（如统计所有评论的平均分）只需读取“评分”这一列，极大降低了I/O开销。

分布式存储

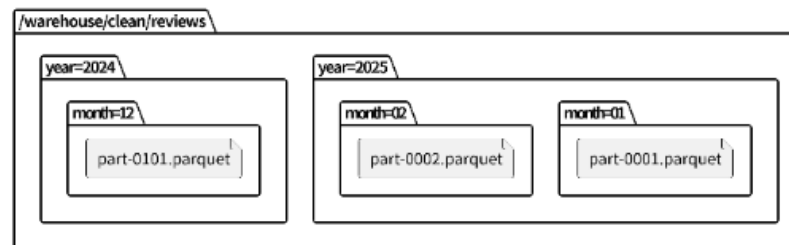
按时间分区



通过按时间分区，当查询“2023年第一季度的评论趋势”时，系统能利用分区裁剪技术，只扫描对应的数据文件，避免无效的全表扫描。

性能优化:

- 使用分区裁剪进行优化



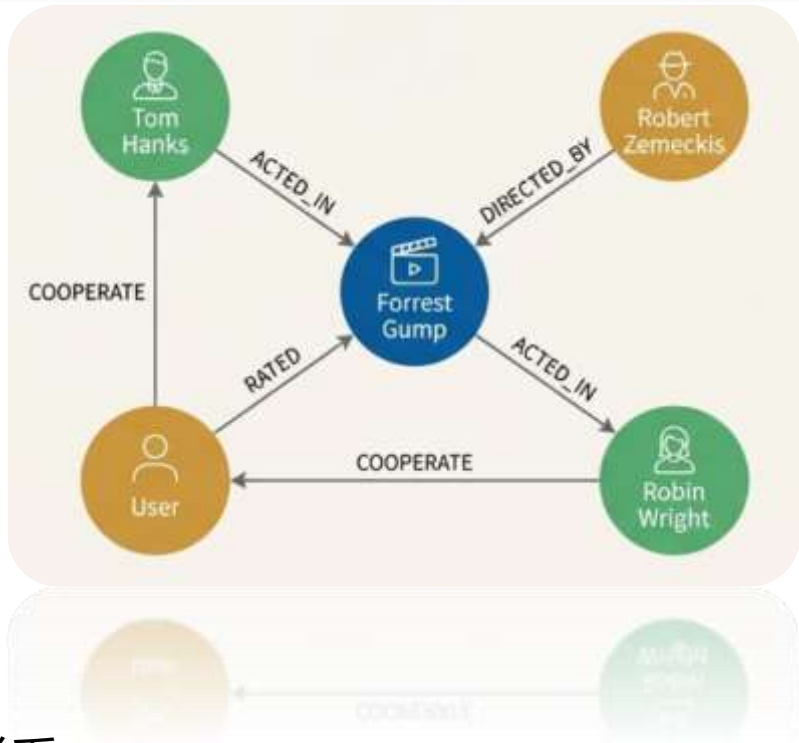
图数据库

节点设计:

用四个节点把电影和评论进行联系

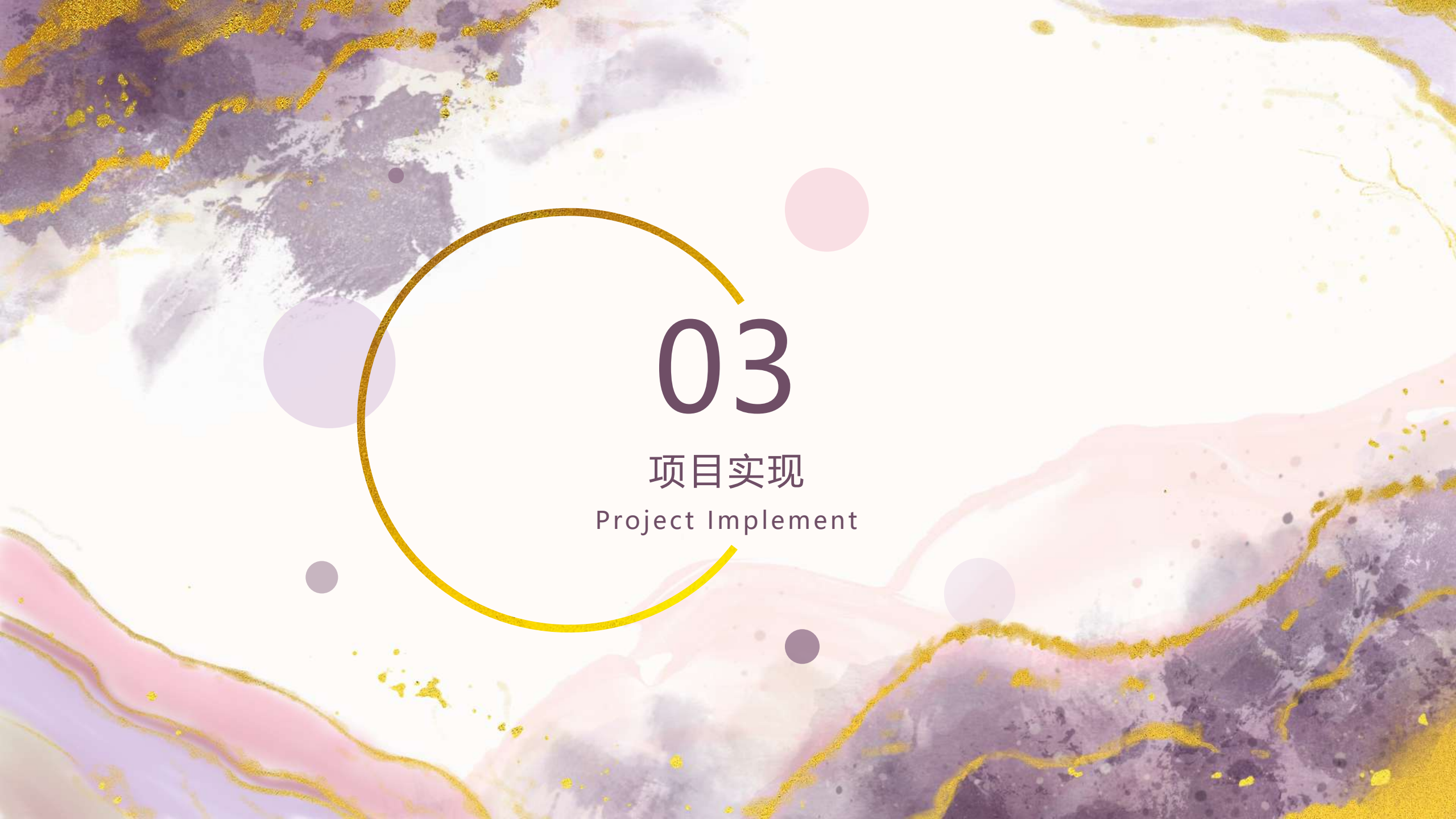
关系类型	起点节点	终点节点	属性	说明
RATED	User	Movie	score, review_time	用户对电影评分, 支持统计评分分布和按时间查询
ACTED_IN	Actor	Movie	role	演员参演电影关系
DIRECTED	Director	Movie		导演执导电影关系
CO_ACTED_WITH	Actor	Actor	times (可选)	演员合作网络关系
COLLABORATED_WITH	Director	Actor	movies_count	导演与演员合作关系

```
CREATE CONSTRAINT IF NOT EXISTS FOR (m:Movie) REQUIRE m.id IS UNIQUE;  
CREATE CONSTRAINT IF NOT EXISTS FOR (a:Actor) REQUIRE a.id IS UNIQUE;  
CREATE CONSTRAINT IF NOT EXISTS FOR (d:Director) REQUIRE d.id IS UNIQUE;  
CREATE CONSTRAINT IF NOT EXISTS FOR (u:User) REQUIRE u.id IS UNIQUE;
```



关系设计:

设计不同节点之间的联系，
将四种不同的身份进行相连



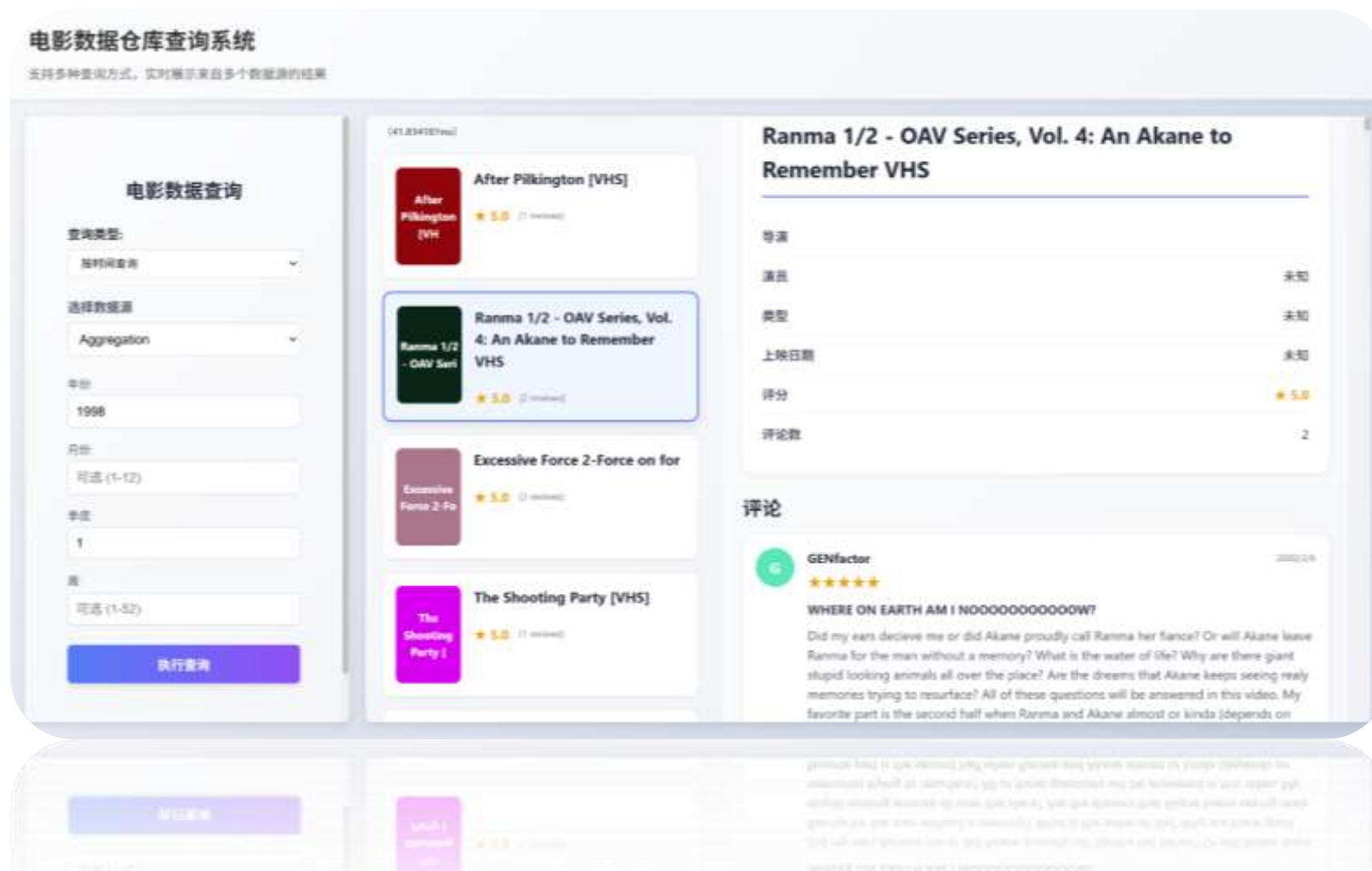
03

项目实施
Project Implement

项目实现

前端页面:

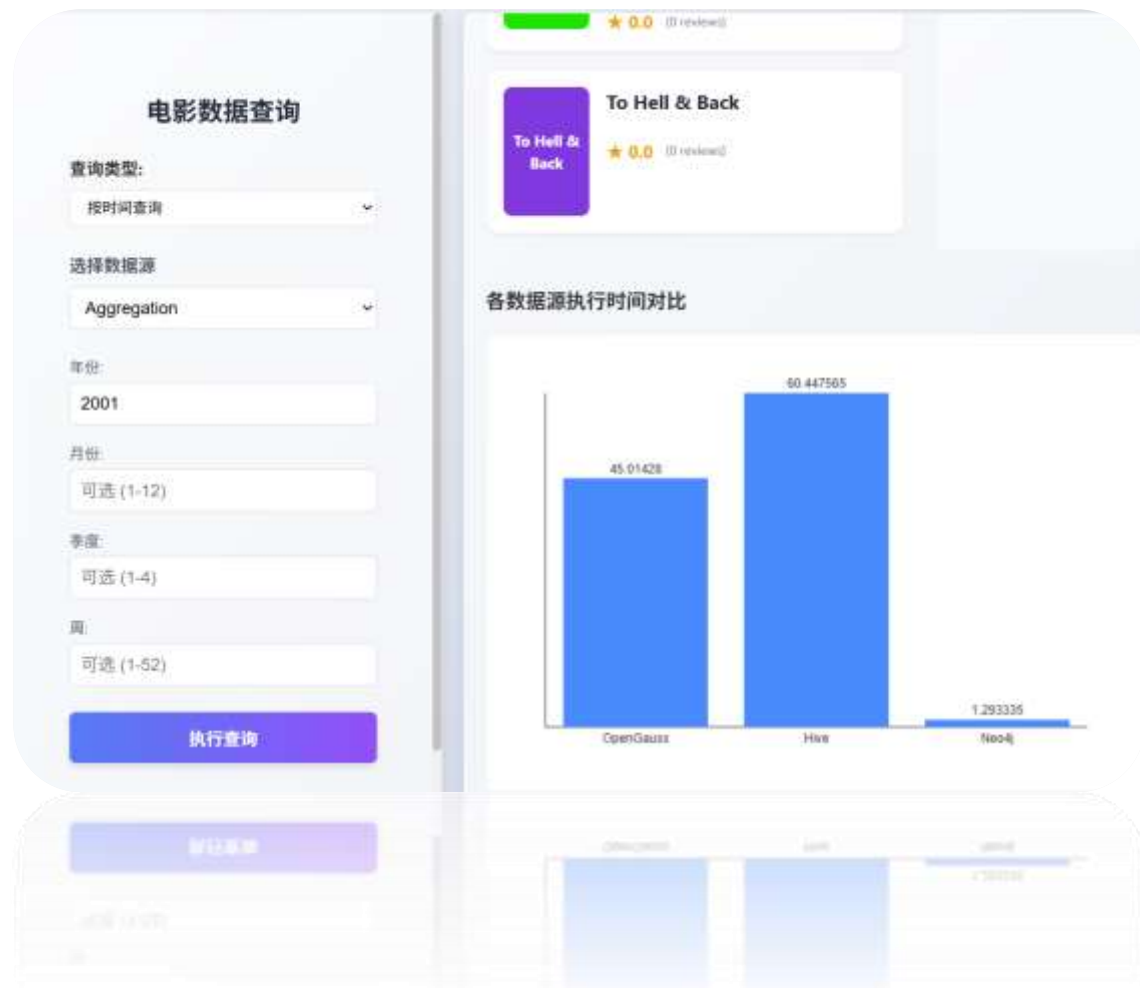
- 页面分为左右两个布局，分别是查询条件界面和查询结果界面
- 查询页面每个电影点进会显示详细信息和评论信息



项目实现

不同数据存储源比较:

- 将采用不同的数据存储来源的查询时间进行比较，得出可视化图画结果
- 可以看出对于几千条数据，节点自带属性，不存在Join的Neo4j的性能就远好于Hive和OpenGauss，而采用了索引和时间维度字段的OpenGauss的性能反超了以年月分布式存储在单查年份上没什么优势的Hive



项目实施

同数据库优化前后比较:

- 利用日志输出，将相同的数据库在优化前后运行相同SQL所需查询时间进行比较。
- 例如，在关系型数据库中，应用时间降序索引前查一条评论所需时间平均40s，应用了时间降序索引之后查一条评论所需时间降到了20ms~40ms

OpenGauss	SUCCESS	40.33s
OpenGauss	SUCCESS	40.47s
OpenGauss	SUCCESS	42.62s
OpenGauss	SUCCESS	42.81s
OpenGauss	SUCCESS	38.09s
OpenGauss	SUCCESS	37.93s



```
>Run  
CREATE INDEX IF NOT EXISTS idx_fact_movie_time ON fact_reviews(movie_id, review_time DESC);  
>Run
```

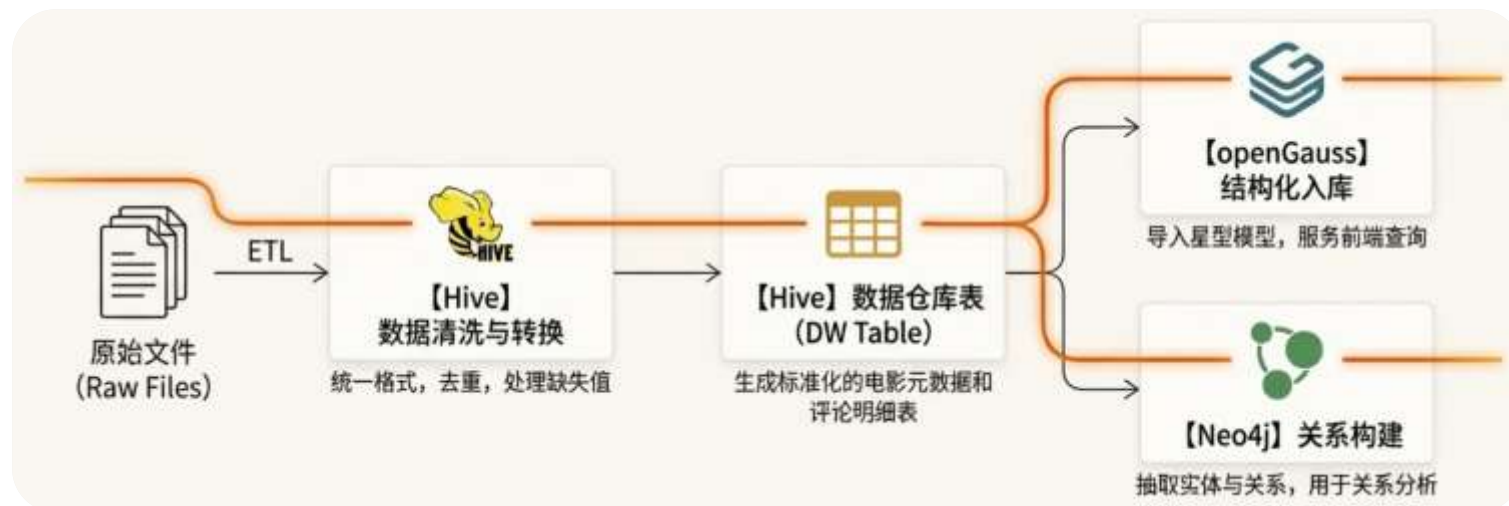


OpenGauss	SUCCESS	0.0474s
OpenGauss	SUCCESS	0.0321s
OpenGauss	SUCCESS	0.0224s
OpenGauss	SUCCESS	0.0241s
OpenGauss	SUCCESS	0.0258s
OpenGauss	SUCCESS	0.0271s
OpenGauss	SUCCESS	0.0235s
OpenGauss	SUCCESS	0.0227s

项目实施

ETL:

对数据进行清洗合并、
提取



溯源查询:

对数据在整个数据处理流程中的**来源、加工路径和去向**进行追踪和定位

==== 总计：共删除 62397 个非电影 ASIN ====

```
{
  "harry_potter_movies": 71,
  "harry_potter_versions": 116,
  "hp1_webpages": 15,
  "hp1_webpages": 12
}
```




04

团队工作

Teamwork

团队工作



版本管理:

- 通过Git + GitHub进行，代码共享以及版本控制
- 版本控制清晰，能进行独立功能的开发



团队分工:

2351299 王雷 33.3%

2354185 周达 33.3%

2351129 黄景胤 33.3%

develop 3 Branches 0 Tags		Go to file	Add file	Code
This branch is 73 commits ahead of and 1 commit behind main				
WingWR Merge branch "feature/backend" into develop b12e5a9 · 12 hours ago 76 Commits				
backend	feat: 完成后端日志记录与查询时间返回	13 hours ago		
doc	del: 删除不必要文件	yesterday		
etl	feat: 初始化前端文件，完成数据库设计和部署参考文档，完...	3 weeks ago		
frontend	feat: 修改查询结果的显示，改成电影和数据左右视图	13 hours ago		
governance	feat: 初始化前端文件，完成数据库设计和部署参考文档，完...	3 weeks ago		
warehouse/docker	fix: 修改建表，符合2核8G服务器	17 hours ago		
.gitignore	fix: 更新.gitignore，对warehouse/docker/hdfs/不跟踪	2 days ago		
README.md	feat: initial README	3 weeks ago		
pyrightconfig.json	fix: 完善了后端涉及到的数据结构，解决了后端报错	3 days ago		



感谢观看

THANK YOU FOR WATCHING

Team:

2351299 王雷

2354185 周达

2351129 黄景胤